CI/CD debugging

A diagnosis is a hypothesis. A rerun is the proof.

Session M4-03 | 3 hours

Agenda

Segment	Time
Pipeline anatomy	10 min
Failure classes	12 min
Copilot as a diagnostician	10 min
Demo: a real failed run	12 min
What "agentic" really means	8 min
Pros, cons, tips, lab handoff	8 min

Pipeline anatomy

```
trigger  ->  workflow  ->  job (on a runner)  ->  step
push, PR     .github/      ubuntu-latest          uses: / run:
             workflows/
```

- **Trigger:** the event that starts a run.
- **Job:** steps that share one **runner** (a fresh VM).
- **Step:** one action or shell command.
- **Context:** data like `github.ref` and `secrets.NAME`.
- **Artifact:** a file a job saves for later jobs or download.

Jobs don't share files unless you upload an artifact.

Five failure classes

Class	Typical log clue
Dependency	`npm ci` can't find or match the lockfile
Environment, secret, permission	empty variable, `Resource not accessible by integration`
Flaky test	passes on rerun, no code change
Matrix	one version fails, others cancelled
Cache	lockfile not found, stale restore

Find the first red step. Later errors are often side effects.

Read a real run

```
gh run list --limit 5
gh run view <run-id> --log-failed
gh run rerun <run-id> --failed
```

Works in the same terminal where Copilot CLI runs.

Three ways to ask Copilot

Where	Use it for
Explain error on a failed check or run page	A quick read of one failure
Copilot CLI in your clone	Read logs, reproduce locally, edit, rerun
Cloud agent	Hand off a fix as an issue and review its pull request

Check current docs: Explain error is described for failed checks and the run summary page.

Hypothesis, then proof

1. Quote the first failing line.
2. Ask for a cause **and** what would disprove it.
3. Reproduce the command locally.
4. Make the smallest fix.
5. Push and read the new run.

A green run proves the fix. A plausible explanation doesn't.

One failure can hide another

```
run 1: secret empty        -> fix: add the secret
run 2: 403 on label create -> fix: add issues: write
```

Fix one cause per push. Read the next log fresh.

What "agentic" really means

Feature	What it does
Copilot Autofix	One suggested fix for a CodeQL alert. You apply it. Free for public repos.
Agentic autofix	Assign a default-branch alert to Copilot. It validates the fix and opens a draft PR. Must be enabled per repo.
Cloud agent	Works an issue you assign to Copilot and opens a PR

None of them merges for you. **You review the PR.**

Review a security patch

- Does it fix the cause or only the alert?
- Does it change behavior for valid input?
- Is there a test that fails without the fix?
- Did it touch unrelated files?

Make the alert disappear and break the feature? That's a failed fix.

Pros, cons, tips

Pros: fast first reading of long logs; suggests the right file to open; drafts the fix.

Cons: guesses when the log is thin; can "fix" by deleting the check; sees only what you give it.

Tips: give it the failed log, not the whole log; ask for one cause at a time; keep fixes on a branch.

Lab

Push a broken sample repo, watch it fail, and fix it with Copilot.